Convolutional Neural Networks (CNN)

Intelligent Systems

Escuela Politécnica de Ingeniería de Gijón

Universidad de Oviedo

Sandra Ranilla-Cortina

ranillasandra@uniovi.es

Overview

What is a Convolutional Neural Network (CNN)?

What is a Convolutional Neural Network (CNN)?

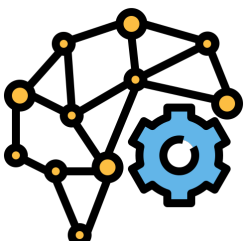
Convolutional Neural Networks, or **CNNs**, are a class of **deep neural networks** specialized in processing grid-like data (e.g., images), but also text and audio.

Inspired by human vision, **CNNs** excel at:

→ **Automatic feature extraction**

→ **Reducing parameters**

→ **Pattern recognition**.



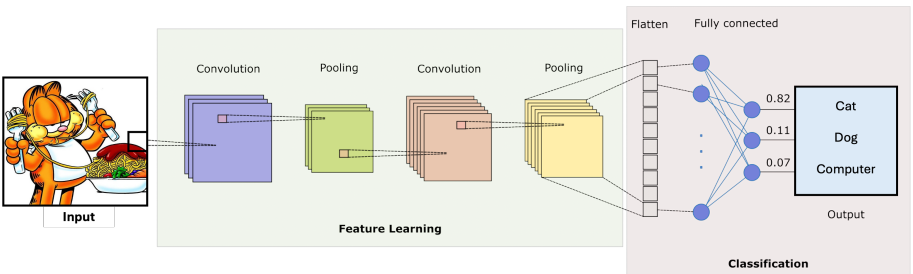
What is a Convolutional Neural Network (CNN)?

CNNs have revolutionized *AI*, achieving state-of-the-art results in tasks like:

→ **Image Recognition**: Facial recognition, medical imaging, self-driving cars

→ **NLP**: Text classification, sentiment analysis

→ **Generative AI**: Deepfakes, artistic style transfer.



How does a CNN recognize a cat? Let's find out!

Basic Concepts of CNNs

Understanding Filters & Kernels

What is a **filter**?

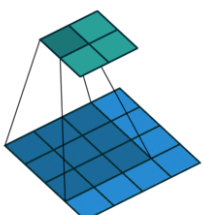
→ A small matrix (**kernel**) used to detect patterns in an image

→ Example: Edge detection, sharpening, blurring.

How does it work?

→ The **filter** slides over the image and performs element-wise multiplication

→ Produces a feature map highlighting detected patterns.



3x3 filter

Convolution Operation

Mathematical formulation for an input x and kernel w:

$$y(i,j) = (x * K)(i,j) = \sum_{m=0}^{m-1} \sum_{n=0}^{n-1} x(i+m, j+n) \cdot w(m,n)$$

where:

- $x(i,j)$ represents the input image pixel value at position (i,j)
- $w(m,n)$ is the filter (**kernel**) value (**weight**) at position (m,n)
- $y(i,j)$ is the output feature map at position (i,j).

Padding & Stride in CNNs

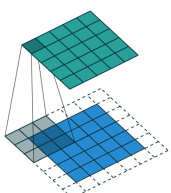
The output shape of the convolutional layer is determined by the shape of the input and the shape of the convolution kernel.

Two important hyper-parameters that affect to the dimensions of y:

Padding

→ Adds extra pixels around the input to preserve spatial dimensions

→ Ensures that when the kernel is applied, the output feature map remains the same size as the original input.



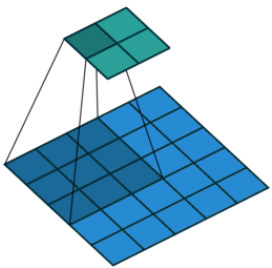
3x3 filter with padding of 1

Padding & Stride in CNN

Stride

→ Controls how much the kernel shifts over the input image

→ Output feature map will have the same spatial dimensions as the input.



3x3 filter with stride of 2

CNN Model Architecture

CNN Model Architecture

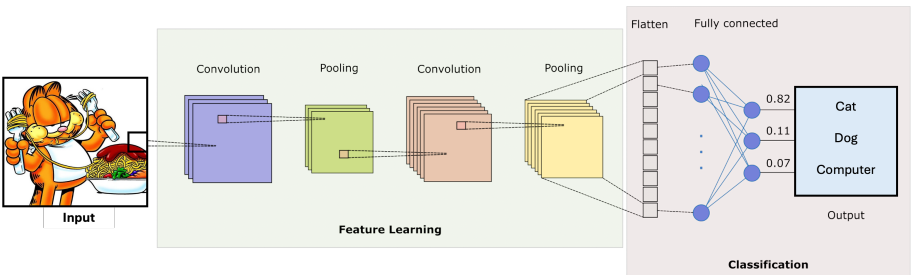
A Convolutional Neural Network (**CNN**) is comprised of four main layers:

i) **Input Layer**

ii) **Convolutional Layer**

iii) **Pooling Layer**

iv) **Fully Connected Layer**.



CNN Model Architecture

Input Layer

The **Input Layer** is where the data is fed into the network:

→ Receives the raw data (e.g., image), and processes it before passing it to the subsequent layers

→ Generally represented as 3D matrices: **height x width x channels** (e.g., 32x32x3 for a color image with 3 channels: red green, and blue)

→ The input data may undergo normalization to improve the network's training efficiency.

Convolutional Layer

The **Convolutional Layer** is responsible for detecting features from the input data, such as edges, textures, and patterns:

→ **Kernels**: Slide over the input image to detect specific features by computing dot products at each position

→ **Feature Maps**: Generate feature maps that highlight the detected features, such as edges or corners

→ **Padding + Stride**: Maintain spatial dimensions + define filters step size

→ **Activation Function**: An activation function (typically **ReLU**) is used to introduce non-linearity into the network.
ReLU keeps positive values as they are and replaces negative values with zero, helping the network learn complex patterns more efficiently.

Pooling Layer

The **Pooling Layer** reduces the spatial dimensions of the feature maps, making the network more efficient and helping to prevent overfitting:

- → **Max Pooling**: The most common operation, where the maximum value from a defined window (e.g., 2x2) is selected, preserving important features
- → **Average Pooling**: Instead of the maximum, it takes the average value from the window, though it's used less frequently.

Fully Connected Layer

The **Fully Connected Layer** is where the network makes final predictions based on the features extracted by the previous layers:

→ Each neuron in the **fully connected layer** is connected to every neuron in the previous layer, forming a **dense network**

→ The output from the previous layers is **flattened** (i.e., converted into a 1D vector) before being passed into the FC layer

→ The FC layer combines the extracted features and performs the final decision-making, **output probabilities**.
Typically using a **sigmoid** (binary) or **softmax** (multi-class) activation function for classification tasks.

CNN Model Training

CNN Model Training

Training a **CNN Model** involves optimizing its weights (parameters) to minimize errors and improve prediction.

Key steps of the **training**:

i) **Forward Propagation**: Input passes through the model → **Predictions**

ii) **Loss Calculation**: Compares predictions with actual labels

iii) **Backpropagation & Optimization**: Computes gradients using the loss function and update weights via **Gradient Descent**

iv) **Iterations & Convergence**: Repeats until loss is minimized and model performance stabilizes

v) **Validation & Regularization**: Evaluates on a validation set to prevent overfitting (Dropout, Batch Normalization, Weight Decay)

vi) **Evaluation**: Performance metrics (accuracy, MSE, etc.).

Parameters in CNN Model Training

Different **parameters** and **hyperparameters** are used at each **training** stage:

i) **Forward Propagation**:

Layer Hyperparameters: Filters (size/number), padding, stride (CONV/POOL), units (FC)

ii) **Loss Calculation**:

Loss Function: Cross-Entropy (classification), MSE (regression)

iii) **Backpropagation & Optimization**:

Trainable Parameters: Weights (CONV, FC layers)
Optimizer: SGD, Adam, etc.
Optimization Hyperparameters: Learning rate, momentum

iv) **Iterations & Convergence**:

Training Hyperparameters: Batch size, number of epochs.

Parameters in CNN Model Training

Different **parameters** and **hyperparameters** are used at each **training** stage:

i) **Forward Propagation**:

Layer Hyperparameters: Filters (size/number), padding, stride (CONV/POOL), units (FC)

ii) **Loss Calculation**:

Loss Function: Cross-Entropy (classification), MSE (regression)

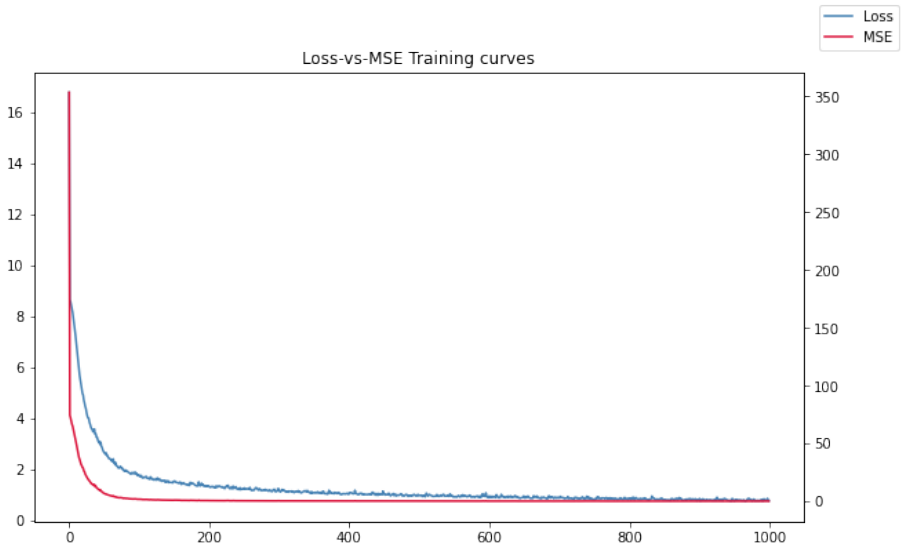
iii) **Backpropagation & Optimization**:

Trainable Parameters: Weights (CONV, FC layers)
Optimizer: SGD, Adam, etc.
Optimization Hyperparameters: Learning rate, momentum

iv) **Iterations & Convergence**:

Training Hyperparameters: Batch size, number of epochs.

CNN Training: Loss & Performance Curves over Epochs



Training Performance of CNN Model

Applications of CNNs

Applications of CNNs

Convolutional Neural Networks (**CNNs**) have a wide range of applications, a few ones:

→ **Image Classification**: Medical imaging, autonomous vehicles

→ **Object Detection**: Facial recognition, security systems

→ **Segmentation**: Medical image analysis, satellite imaging

→ **NLP**: Sentiment analysis, voice assistants

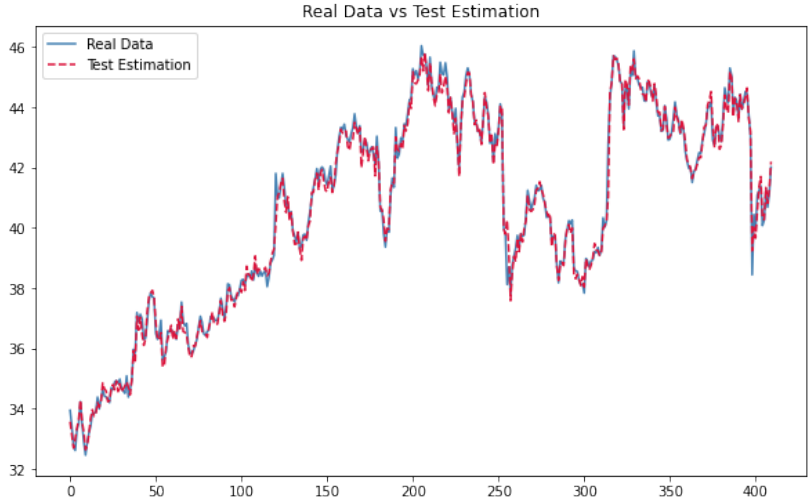
→ **Time Series Forecasting**: Stock prices, weather forecasting

→ **Video Analysis**: Surveillance, autonomous driving

→ **Generative Models**: Style transfer, image super-resolution, video games

→ **Anomaly Detection**: Machinery failure detection, credit card fraud.

Time Series Forecasting - Stock prices



CNN Model - Stock Market Price Estimation

Generative Models - Style Transfer

CNN Model - Style Transfer

NLP - Sentiment analysis

CNN Model - Sentiment Analysis

References

References i

[1] Bishop, C. M., and Nasser, M. N. *Pattern recognition and machine learning*. New York: springer, 2006.

[2] Charu, C. A. *Neural networks and deep learning*. Cham: Springer, 2018.

[3] Chollet, F. *Deep learning with Python*. Simon and Schuster, 2021.

[4] Deisenroth, M. P., Faisal, A. A., and Ong, C. S. *Mathematics for machine learning*. Cambridge University Press, 2020.

[5] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. *Deep learning*. Cambridge: MIT press, 2016.

[6] Géron, A. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Inc., 2022.

[7] López de Prado, M. *Advances in Financial Machine Learning*. John Wiley & Sons, 2018.

[8] Raschka, S., and Mirjalili, V. *Python machine learning: Machine learning and deep learning with Python, , scikit-learn, and TensorFlow 2*. Packt publishing ltd, 2019.